Лабораторний практикум 4
Продуктовий беклог та методи пріоритизації

Мета роботи: Сформувати практичні навички написання User Stories з коректними Acceptance Criteria, побудови ієрархічного беклогу та застосування кількісних і якісних методів пріоритизації.

Теоретичні відомості

1. Ієрархія беклогу

Продуктовий беклог організований ієрархічно від найбільших до найменших одиниць роботи. Ця структура допомагає команді мати одночасно стратегічний погляд (Epic) та тактичний план (Task).

Product Backlog – продуктовий беклог – впорядкований список всіх відомих вимог, покращень, виправлень та завдань, що потрібно виконати для розвитку продукту. Беклог є власністю Product Owner.

Ознаки здорового беклогу:

– Detailed appropriately (Деталізований відповідно): Items вгорі детально описані і готові до розробки, нижні – схематичні

– Emergent (Живий): регулярно оновлюється, нові Items додаються, неактуальні видаляються

– Estimated (Оцінений): Items вгорі оцінені в Story Points, нижні можуть бути без оцінки

– Prioritized (Пріоритизований): Items відсортовані за цінністю, ризиком та складністю

Backlog Grooming (або Backlog Refinement): регулярна зустріч PO та команди для уточнення, декомпозиції та оцінки Items. Рекомендована частота: раз на тиждень, 1–2 години.

Epic (Епік) – великий блок роботи, що охоплює кілька спринтів та описує значну функціональну область продукту або бізнес-задачу. Epic – контейнер для пов'язаних Features та User Stories.

Epic відповідає на питання: «Яку велику задачу ми вирішуємо для користувача?»

Ознаки гарного Epic:

- ✓ прив'язаний до конкретного JTBD або стратегічної цілі;
- ✓ має чітку бізнес-цінність та вимірюваний результат;
- ✓ може бути розбитий на менші Stories;
- ✓ занадто великий для одного Sprint.

Jira Epic: зазвичай відображається кольоровою міткою та об'єднує всі Stories однієї теми.

Time frame: 1–3 місяці виконання.

Feature (функціональність) – середній рівень ієрархії між Epic та User Story. Feature описує конкретну можливість або набір можливостей продукту, що має самостійну цінність для користувача.

Feature відповідає на питання: «Яку конкретну МОЖЛИВІСТЬ ми надаємо?»

Ознаки гарної Feature:

- ✓ самостійна цінність для користувача (можна демонструвати окремо);
- ✓ вписується в один-два Sprint;
- ✓ складається з 3–8 User Stories;
- ✓ прив'язана до батьківського Epic.

Feature flag (фіча-флаг): технічний механізм для вмикання/вимикання Feature для конкретних користувачів без деплою нового коду. Дозволяє робити A/B тести та поступові rollouts.

Приклад: Feature «Автоматична категоризація витрат»: входить до Epic «Система фінансового обліку». Складається з 4–5 User Stories: категоризація за MCC-кодом, ручна зміна категорії, правила категоризації, звіт по категоріях.

КОПІЯ

User Story (Користувацька історія)

Найменша одиниця функціональної роботи в беклозі, що описує потребу з точки зору конкретного користувача. Це не технічне завдання – це опис того, чого хоче користувач та навіщо.

Стандартний формат (шаблон Майка Коена):

«Як [роль/Persona], я хочу [дія/можливість], щоб [бізнес-вигода/ціль]»

User Story не є:

- ✓ Технічним завданням: «Додати кнопку в базу даних» – це Task
- ✓ Системною вимогою: «Система повинна підтримувати 1000 запитів/сек» – це Non-Functional Requirement
- ✓ Фічею: Story повинна бути виконана за один Sprint

Criteria for good story – INVEST:

- ✓ **Independent (незалежна):** мінімальні залежності від інших Stories
- ✓ **Negotiable (обговорювана):** деталі обговорюються, не зафіксовані в камені
- ✓ **Valuable (цінна):** має цінність для користувача або бізнесу
- ✓ **Estimable (оцінювана):** команда може оцінити в Story Points
- ✓ **Small (маленька):** виконується за Sprint
- ✓ **Testable (тестована):** можна перевірити чи виконана

Приклад: Погана Story: «Реалізувати REST API endpoint для транзакцій» (технічна задача, не цінність). Гарна Story: «Як фрілансер Дмитро, я хочу бачити список всіх транзакцій за поточний місяць, щоб розуміти свої витрати без необхідності відкривати банківський застосунок».

Найчастіші помилки при написанні User Stories:

- ✓ **Технічне формулювання:** «Як система, я хочу записати транзакцію в базу даних» – неправильно. Story – від імені людини.
- ✓ **Відсутність вигоди:** «Як користувач, я хочу бачити список транзакцій» – де «щоб...»? Без мети незрозуміла цінність
- ✓ **Занадто велика: якщо Story не виконується за Sprint –** обов'язково розбийте на кілька менших
- ✓ **Включення деталей реалізації:** Story не повинна диктувати технічне рішення – лише ЩО потрібно
- ✓ **Відсутність прив'язки до JTBD:** кожна Story повинна відповідати на питання «Яку роботу виконує цей користувач?»

Acceptance Criteria (AC) – Критерії Приймання

Набір умов, яким повинен відповідати продукт, щоб User Story вважалась завершеною та прийнятою Product Owner. AC – це контракт між PO та командою.

Формат Given-When-Then (Gherkin syntax – від Behaviour Driven Development):

- ✓ **Given (Дано): передумова –** стан системи або контекст перед дією. «Дано, що користувач авторизований та має 5 транзакцій за поточний місяць»
- ✓ **When (Коли):** дія, яку виконує користувач або подія в системі. «Коли він відкриває розділ 'Транзакції цього місяця'»
- ✓ **Then (Тоді):** очікуваний результат або поведінка системи. «Тоді він бачить список з 5 транзакцій, відсортованих за датою від новіших до старіших»

Правила написання AC:

1) Мінімум 3 AC для кожної Story
2) **Кожен AC –** окремий тест-кейс
3) Написані перед розробкою (BDD approach)
4) **Тестовані:** чітко «виконано» або «не виконано»
5) **Покривають:** happy path + edge cases + error cases

Приклад: Story: «Як фрілансер, я хочу бачити залишок бюджету, щоб знати скільки можу ще витратити». AC#1: Given користувач встановив місячний бюджет 10 000 грн та витратив 3 500 грн, When він відкриває головний екран, Then він бачить «Залишок: 6 500 грн (65%)». AC#2: Given

КОПІЯ

залишок менше 20% бюджету (тобто менше 2 000 грн), When він відкриває головний екран, Then залишок відображається червоним кольором з попередженням. AC#3: Given бюджет не встановлено, When він відкриває головний екран, Then він бачить кнопку «Встановити місячний бюджет» замість індикатора залишку.

Практичний лайфхак для написання AC:

Задайте собі ці питання для кожної Story:

1. Що відбувається в «нормальному» сценарії (happy path)?
2. Що відбувається, якщо дані відсутні або порожні?
3. Що відбувається при помилці або невалідних даних?
4. Як реагує система при повільному інтернеті?
5. Що бачить перший раз зареєстрований vs досвідчений користувач?

Відповіді на ці питання = ваші AC. Кожен сценарій = окремий Given-When-Then.

Definition of Done (DoD) – Визначення «Готово»

Командний стандарт якості – повний перелік умов, при яких будь-яка User Story вважається «зробленою». DoD є глобальним для всіх Stories і не дублює Acceptance Criteria конкретної Story.

DoD vs AC:

- ✓ **DoD:** «Чи відповідає ця Story нашим стандартам якості?» (однаково для всіх)
- ✓ **AC:** «Чи відповідає ця Story вимогам цієї конкретної функціональності?» (унікально для кожної)

Типові пункти DoD для мобільного застосунку:

1. Код написаний та пройшов code review (мінімум 1 reviewer)
2. Unit-тести написані та проходять (coverage ≥ 80%)
3. Функціональність протестована QA-інженером на test середовищі
4. Відсутні критичні та high-priority баги
5. Функціональність задеплоєна на staging середовище
6. Product Owner підтвердив відповідність всім AC
7. Документація оновлена (якщо потрібно)
8. Performance перевірено (час завантаження < 2 сек)

DoD – це командна відповідальність. SM допомагає команді дотримуватись DoD.

Приклад: Без DoD: розробник каже «я закінчив», QA знаходить 10 багів, PO відхиляє Story на Review. З DoD: команда знає точні критерії, Story не вважається готовою, поки всі пункти не виконані.

Task (Технічна задача)

Найменша одиниця роботи в Scrum, що представляє конкретний технічний крок реалізації User Story. Tasks створюються командою розробки під час Sprint Planning.

Task відповідає на питання: «ЯК саме ми реалізуємо цю Story?»

Ознаки гарного Task:

- ✓ Виконується за 2–8 годин одним розробником
- ✓ **Технічно конкретний:** «Написати API endpoint GET /transactions», «Написати unit-тести для TransactionService», «Зробити UI компонент TransactionList»
- ✓ Призначений конкретному виконавцю

WELL-DECOMPOSED STORY: 3–7 Tasks:

- ✓ Якщо Tasks менше 3 – Story може бути занадто маленькою або занадто спрощеною.
- ✓ Якщо більше 10 – Story треба розбити.

Приклад: Story «Показати список транзакцій за місяць» → Tasks:

Task 1: «GET /api/transactions?month= endpoint», Task 2: «TransactionList React-компонент», Task 3: «Unit-тести для TransactionRepository», Task 4: «Integration-тест API endpoint», Task 5: «Pagination компонент». Разом ≈ 12 год.

КОПІЯ

Рівень	Назва	Приклад	Виконується за	Типова оцінка
1 (найбільший)	Epic	«Фінансовий облік»	2–4 місяці	T-shirt: XL/XXL
2	Feature	«Автокатегоризація»	1–2 спринти	Story Points: 13–34
3	User Story	«Бачити список транзакцій»	1 Sprint (1–2 тижні)	Story Points: 1–13
4 (найменший)	Task	«GET /transactions endpoint»	2–8 годин	Години

2. Оцінка складності в Agile

Story Points – Сторі-поінти

Відносна одиниця вимірювання складності, зусиль та невизначеності User Story. Story Points не відповідають годинам чи дням.

SP вимірюють три виміри одночасно:

- ✓ **Complexity (Складність):** наскільки технічно складно реалізувати
- ✓ **Effort (Зусилля):** скільки роботи потрібно зробити
- ✓ **Uncertainty (Невизначеність):** наскільки добре ми розуміємо що робити

Чому не години? Одна й та сама Story займе 2 год у senior-розробника і 8 год у junior. SP – командна оцінка відносної складності, не часу конкретної людини.

Шкала Фібоначчі: 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ∞ (або ?, якщо Story взагалі незрозуміла)

Чому Фібоначчі? Різниця між сусідніми числами відображає невизначеність оцінки: різниця між 1 і 2 – мала і значуща, різниця між 20 і 21 – практично несуттєва. Велика невизначеність = велика різниця між оцінками.

Приклад: Story «Кнопка поширення» = 1 SP (проста, добре зрозуміла). Story «Інтеграція з API Monobank» = 13 SP (складна технічно, є невизначеності). Story «Впровадити AI-рекомендації» = 34 або ? (висока невизначеність → краще розбити).

Planning Poker (Покер Планування)

Метод групової оцінки Story Points, що запобігає груповому тиску та виявляє різне розуміння задачі. Назва – бо схожий на карткову гру.

Матеріали: набір карток для кожного учасника зі значеннями Фібоначчі (1, 2, 3, 5, 8, 13, 21, ?, ∞) або онлайн-інструмент (Miro Planning Poker, PlanITPoker, Scrum Poker Cards).

Процес Planning Poker:

1. PO зачитує та пояснює User Story. Команда ставить уточнюючі питання
2. Кожен учасник мовчки вибирає картку зі своєю оцінкою
3. За командою (або лічилкою «3-2-1-reveal!») всі відкривають картки ОДНОЧАСНО
4. Якщо всі оцінки однакові – приймається. Якщо розходяться – обговорення
5. Учасники з найменшою та найбільшою оцінками пояснюють свою логіку
6. Повторна оцінка → Консенсус

Чому одночасно? Якщо один учасник показує картку раніше – інші підлаштовуються. Одночасне розкриття – ключовий принцип.

Ознаки проблем: якщо завжди одна людина говорить першою і всі погоджуються – це не Planning Poker, а ілюзія консенсусу.

Velocity – Швидкість команди

Середня кількість Story Points, яку команда стабільно виконує за один Sprint. Velocity – це метрика для планування, а не ціль для максимізації.

Як розраховується: sum of completed SP за останні 3–5 Sprints → середнє значення.

Важливі правила:

КОПІЯ

- ✓ Velocity вимірюється лише для завершених Stories (ті, що відповідають DoD)
- ✓ Не порівнюйте Velocity різних команд (залежить від розміру команди, складності, DoD)
- ✓ Velocity стабілізується після 3–5 спринтів
- ✓ Нормальне коливання: ±20% між спринтами
- ✓ Velocity може падати при: нових членах команди, технічному боргу, зовнішніх змінах

Чому не максимізувати Velocity? Якщо команда «оптимізує» SP (занижує оцінки, щоб виглядати продуктивнішою) – втрачається сенс оцінки. Velocity – не KPI для преміювання.

Приклад: Sprint 1 = 28 SP, Sprint 2 = 34 SP, Sprint 3 = 31 SP, Sprint 4 = 33 SP. Середня Velocity = 31.5 SP. На Sprint 5 команда бере 30–34 SP (з невеликим запасом).

3. Методи пріоритизації

Пріоритизація – це мистецтво та наука прийняття рішень в умовах обмежених ресурсів. Жоден метод не є «правильним» сам по собі – кожен підходить для певного контексту та питання. Досвідчений PM комбінує кілька методів.

MoSCoW Prioritization

Популярний фреймворк для приоритизації вимог і завдань у проєкті, який ділить усю роботу на чотири чіткі категорії. **Техніка класифікації вимог за ступенем обов'язковості. Назва –** акронім чотирьох категорій:

- ✓ **Must Have (M –** Обов'язково): без цього продукт не може бути випущений. Без цих функцій продукт не вирішує ключову проблему або порушує закон/безпеку. Тест Must Have: «Якщо цього немає – реліз скасовується?» Якщо ТАК → Must Have.
- ✓ **Should Have (S –** Варто мати): важливі функції з великим впливом, але без яких можна запустити першу версію. Будуть реалізовані якщо вистачить часу і ресурсів. Зазвичай можуть бути відкладені на наступний реліз без критичних наслідків.
- ✓ **Could Have (C –** Можна мати): бажані функції, що покращують UX, але мають мінімальний вплив при їх відсутності. «Nice-to-have». Видаляються першими при нестачі часу.
- ✓ **Won't Have (W –** Не будемо мати): свідоме рішення НЕ включати зараз. Це не означає «ніколи» – лише «не в цьому релізі». Важливо: Won't Have допомагає команді фокусуватись та уникати scope creep.

Правило MoSCoW: Must Have не повинен перевищувати 60% загального обсягу Sprint/релізу. Якщо більше – щось має стати Should Have.

RICE Score

Кількісний фреймворк пріоритизації для порівняння Stories або фіч на основі чотирьох параметрів. Розроблений командою Intercom для подолання суб'єктивності.

Формула: RICE = (Reach × Impact × Confidence) / Effort

- ✓ **REACH (Охоплення):** скільки користувачів/клієнтів зачепить ця фіча за один місяць. Вимірюється в ЧИСЛАХ (не відсотках). Джерела: дані аналітики, результати опитування ПР №01.
- ✓ **IMPACT (Вплив):** наскільки сильно ця фіча вплине на ключову метрику (наприклад, конверсія, retention). ШКАЛА: 3 = масивний, 2 = значний, 1 = помірний, 0.5 = мінімальний, 0.25 = несуттєвий.
- ✓ **CONFIDENCE (Впевненість):** наскільки ви впевнені в оцінках Reach та Impact. ШКАЛА: 100% = є кількісні дані, 80% = є результати якісних досліджень (ваше ПР №01), 50% = є певна інформація, 20% = здогадка.
- ✓ **EFFORT (Зусилля):** загальні зусилля команди в людино-місяцях. Сума часу всіх залучених: розробники + дизайнери + QA.

Чим вище RICE Score – тим вищий пріоритет.

Як визначити параметри RICE:

Reach: якщо у вас 1 000 цільових користувачів і 40% використовуватимуть функцію → Reach=400. Для onboarding-функцій Reach = кількість нових реєстрацій/місяць.

КОПІЯ

Impact: подивіться на бізнес-метрику. «Зменшить Churn» → Impact=2. «Покращить дизайн» → Impact=0.5.

Confidence: 100% лише якщо є A/B тест. Результати ваших інтерв'ю → 80%. Тільки відчуття → 20%.

Effort: не займайтесь гіперточністю. «0.25 = 1 тиждень», «0.5 = 2 тижні», «1 = місяць».

Kano Model – Модель Кано

Теорія задоволеності клієнтів, класифікує функції продукту за їх впливом на задоволеність залежно від рівня реалізації.

П’ять категорій Кано:

- ✓ **Must-be (Базові / «Гігієнічні»):** відсутність цих функцій різко знижує задоволеність (незадоволення), але їх наявність не підвищує (це «само собою зрозуміло»). Приклад: безпечний вхід в банківський застосунок – якщо його немає, клієнт іде. Але якщо він є – клієнт просто не незадоволений.
- ✓ **Performance (Лінійні / «Одновимірні»):** прямий лінійний зв'язок: чим краще реалізовано – тим більш задоволений клієнт. Постійне покращення Performance приносить постійне підвищення задоволеності. Приклад: швидкість завантаження застосунку.
- ✓ **Delighter (Захоплюючі / «Атрактивні»):** несподівані функції, що приємно дивують. Відсутність не дратує (клієнт не знав, що це можливо), але наявність викликає захват і лояльність. Приклад: застосунок автоматично аналізує витрати та каже «Ви можете заощадити X, якщо...»
- ✓ **Indifferent (Байдужі):** клієнт не помічає ні наявності, ні відсутності. Розробка таких функцій – марна трата ресурсів.
- ✓ **Reverse (Зворотні):** частина клієнтів незадоволена наявністю функції. Наприклад, просунутий користувач незадоволений «спрощеним» інтерфейсом.

Важливо пам’ятати: з часом Delighters стають Performance, а Performance – Must-be. Колись «вбудований GPS» у смартфоні був Delighter, сьогодні Must-be.

Як провести Kano-опитування:

Для кожної функції ставте два запитання:

1. FUNCTIONAL: «Як ви почуватиметесь, якщо ця функція БУДЕ в продукті?»
2. DYSFUNCTIONAL**:** «Як ви почуватиметесь, якщо цієї функції НЕ БУДЕ в продукті?»

Варіанти відповідей:

1. Дуже задоволений
2. Так і має бути
3. Байдуже
4. Можу прийняти
5. Дуже незадоволений

Таблиця класифікації:

Functional↓ \ Dysfunctional→	Like	Expect	Neutral	Live	Dislike
Like	Q	A	A	A	O
Expect	R	I	I	I	M
Neutral	R	I	I	I	M
Live	R	I	I	I	M
Dislike	R	R	R	R	Q

A=Attractive(Delighter), M=Must-be, O=Performance(One-dimensional), I=Indifferent, R=Reverse, Q=Questionable

ICE Score

Спрощена альтернатива RICE для швидкої пріоритизації без урахування охоплення. Зручна для невеликих команд або коли дані про Reach відсутні.

$$ICE = Impact \times Confidence \times Ease$$

Копія

- ✓ IMPACT (Вплив): 1–10. Наскільки сильно впливає на ключову бізнес-метрику або задоволеність клієнта.
- ✓ CONFIDENCE (Впевненість): 1–10. Наскільки ви впевнені в оцінці Impact. Базується на даних, а не на відчуттях.
- ✓ EASE (Легкість реалізації): 1–10. 10 = дуже легко (1–2 дні), 1 = дуже складно (кілька тижнів).

Переваги ICE: простота, швидкість, не потребує детальних даних.

Недоліки ICE: не враховує масштаб аудиторії (маленька high-Impact фіча може обігнати велику low-Impact).

Opportunity Scoring – Оцінка можливостей

Метод для виявлення незадоволених потреб ринку. Базується на принципі: люди «купують» результати, а не функції.

Алгоритм:

1. Визначте 10–15 Outcomes (бажаних результатів) від вашої ЦА: не «функції» а «що хочуть досягти»
2. Попросіть представників ЦА оцінити кожен Outcome за двома шкалами 1–10:
   - ✓ **IMPORTANCE:** «Наскільки важливо для вас досягти цього результату?» (1=зовсім неважливо, 10=критично важливо)
   - ✓ **SATISFACTION:** «Наскільки добре поточні рішення (будь-які) досягають цього результату?» (1=зовсім не задовольняють, 10=повністю задовольняють)
3. Розрахуйте Opportunity Score:

OPPORTUNITY = Importance + max(Importance – Satisfaction, 0)

Максимальне значення: 20 (Importance=10, Satisfaction=0)

4. Нанесіть результати на матрицю Importance (Y) vs Satisfaction (X).

Зони матриці:

- ✓ UNDERSERVED (High Importance + Low Satisfaction, Opp > 12): ваша ЗОЛОТА можливість
- ✓ CONTESTED (High Importance + Medium Satisfaction, Opp 8–12): конкурентна зона
- ✓ OVERSERVED (High Importance + High Satisfaction): ринок насичений, не витрачайте зусилля
- ✓ NOT IMPORTANT (Low Importance): байдуже, незалежно від задоволеності

КОПІЯ

Завдання

Завдання 1. Побудова ієрархічного беклогу

Відкрийте результати практичних робіт №01–03. Кожна User Story **повинна** бути пов'язана з конкретним JTBD або Pain. Без цього зв'язку Story не має обґрунтування у беклозі.

1. Визначення Epic-ів (3–4 шт.)

Для вашого продукту визначте 3–4 Epic. Кожен Epic повинен:
– відповідати одному з ключових JTBD або стратегічній цілі;
– мати чіткий вимірюваний Business Value;
– бути достатньо великим, щоб охоплювати 5–8 Features.

Epic ID	Назва Epic	Пов'язаний JTBD (з ПР №01–02)	Business Value (яку метрику покращує)	Пріоритет
E-01				High
E-02				High
E-03				Med
E-04				Low

Приклад Epic для фінансового застосунку для фрілансерів:
E-01: «Фінансовий Дашборд» → JTBD: «Розуміти фінансовий стан без зайвих зусиль» → Value: підвищує D7 Retention
E-02: «Синхронізація з банком» → JTBD: «Не витрачати час на ручне введення» → Value: підвищує Activation Rate

2. Features для кожного Epic (6–9 шт.)

Для кожного Epic визначте 2–3 Features. Кожна Feature – самостійна цінність, яку можна продемонструвати окремо.

Feature ID	Назва Feature	Батьківський Epic	User Value (один рядок)	Story Count	Оцінка SP
F-01		E-01			
F-02		E-01			
F-03		E-02			
F-04		E-02			
F-05		E-03			
F-06		E-03			
F-07		E-04			
F-08		E-04			

КОПІЯ

3. Написання User Stories (25+ шт.)

Напишіть мінімум 25 User Stories (≈3 Stories на Feature). Для кожної Story використовуйте повний шаблон нижче. Перші 5 Stories заповніть максимально детально.

шаблон USER STORY

Поле	Що вказати
ID (наприклад US-001)	*Унікальний ідентифікатор. Використовуйте наскрізну нумерацію.*
Feature	*ID та назва батьківської Feature (наприклад F-02 «Синхронізація Monobank»)*
User Story	*«Як [роль/Persona з ПР №01], я хочу [конкретна дія/можливість], щоб [вимірювана вигода/ціль]»*
Тип	*Нова функція / Покращення / Технічний борг / Баг / Документація*
Пов'язаний JTBD або Pain	*Точне формулювання з ПР №01–02. Обов'язково!*
Story Points (за шкалою Фібоначчі)	*1 / 2 / 3 / 5 / 8 / 13 / 21 / ?*
Пріоритет (MoSCoW) з поясненням вибору	*Must Have / Should Have / Could Have / Won't Have*
Acceptance Criteria #1	*Given [передумова/контекст], When [дія], Then [очікуваний результат]*
Acceptance Criteria #2	*Given..., When..., Then...*
Acceptance Criteria #3	*Given..., When..., Then... (edge case або error case)*
Definition of Done	*Загальний DoD вашої команди – однаковий для всіх Stories*
Нотатки / Обмеження / Залежності	*Технічні залежності, edge cases, обмеження платформи, UX-нотатки*

КОПІЯ

Завдання 2. Пріоритизація методом RICE

1. Калібрування параметрів RICE

Перед розрахунком RICE визначте базові параметри вашого продукту:

Параметр для RICE	Ваше значення та обґрунтування
Загальна кількість цільових користувачів на місяць (ринок)	*Джерело: дані з ПР №01 або ринковий аналіз*
Ключова метрика, яку оптимізуємо (Impact target)	
Поточне baseline значення ключової метрики	%
Середній Effort на 1 Story Point (людино-дні)	
Розмір команди (розробники + дизайнери + QA)	*Кількість людей*

2. Таблиця RICE для 15 Stories

Оберіть 15 найважливіших User Stories (по 1–2 від кожної Feature + додаткові). Розрахуйте RICE Score для кожної. Відсортуйте від найвищого до найнижчого.

ID	Назва User Story	Reach (users/міс)	Impact (0.25–3)	Conf. (%)	Effort (люд-міс)	RICE Score	Ранг
US-001							
US-002							
US-003							
US-004							
US-005							
US-006							
US-007							
US-008							
US-009							
US-010							
US-011							
US-012							
US-013							
US-014							
US-015							

3. Аналіз результатів RICE

Після заповнення таблиці дайте розгорнуті відповіді на чотири питання:

Аналіз результатів RICE

Поле	Що вказати
Топ-5 Stories за RICE Score	*Що між ними спільного? Які параметри найбільше вплинули на рейтинг?*
Несподівані результати	*Чи є Stories, де RICE-ранг відрізняється від вашої початкової інтуїції? Чому?*
Stories з низьким Confidence	*Для яких Stories оцінки ненадійні? Що потрібно зробити для підвищення впевненості?*
Обмеження RICE для вашого продукту	*Які фактори важливі для вашого продукту, але не враховані у формулі RICE?*

КОПІЯ

Завдання 3. Kano-аналіз

1. Підготовка Kano-опитування

Оберіть 10 функцій продукту для аналізу (суміш Must Have, Should Have та Could Have з вашого беклогу). Для кожної функції сформулюйте два запитання:

#	Назва функції	Functional запитання (якщо функція є)	Dysfunctional запитання (якщо функції немає)
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

Формулювання запитань для Kano:
Functional: «Як ви почуватиметесь, якщо [функція] буде в застосунку?»
Dysfunctional: «Як ви почуватиметесь, якщо [функція] не буде в застосунку?»
Варіанти відповідей (однакові для обох):
1 = Дуже задоволений
2 = Так і має бути
3 = Байдуже
4 = Можу прийняти
5 = Дуже незадоволений
Мінімум: 10 респондентів – реальних представників вашої ЦА

2. Результати та класифікація

Функція	Must-be (M)	Performance (O)	Delighter (A)	Indifferent (I)	Reverse (R)	Quest. (Q)	Фінальна категорія	Рекомендація
Функція 1								
Функція 2								
Функція 3								
Функція 4								

КОПІЯ

Функція 5								
Функція 6								
Функція 7								
Функція 8								
Функція 9								
Функція 10								

Як визначити фінальну категорію при розбіжності відповідей:

- ✓ Підрахуйте відсоток кожної категорії по всіх респондентах
- ✓ Найбільш представлена категорія – фінальна
- ✓ При рівній кількості: M > O > A > I
- ✓ Q (questionable) відповіді ігноруйте – вони некоректні

3. Висновки Kano

Висновки Kano-аналізу

Поле	Що вказати
Must-be функції (таблиця показала обов'язковість)	
Performance функції (конкурентна перевага)	
Delighter функції (WOW-фактори)	
Indifferent функції (не витрачати ресурси)	
Порівняння з RICE	
Стратегічний висновок для MVP	

КОПІЯ

Завдання 4. Opportunity Scoring

1. Формулювання Outcomes

Перефразуйте потреби користувача у форматі «бажаного результату» (Outcome). Outcome – це що хоче досягти користувач, а не яким способом. Формат: Дієслово + Об'єкт + Контекст.

Погано: «Хочу синхронізацію з Monobank»
Добре: «Отримати актуальний баланс без ручного оновлення»
Погано: «Хочу статистику»
Добре: «Зрозуміти, на що витрачаю найбільше грошей щомісяця»

2. Збір оцінок та розрахунок

Зберіть оцінки від 10–15 представників ЦА. Для кожного Outcome – Importance та Satisfaction (шкала 1–10). Розрахуйте Opportunity Score:

#	Outcome (бажаний результат)	Import. (сер.)	Satisf. (сер.)	Opp. Score	Зона	Пріоритет
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						

Формула: Opportunity = Importance + max(Importance − Satisfaction, 0)
Максимум: 20 (якщо Importance=10, Satisfaction=0)
Зони:

- ✓ Underserved (Score > 12): висока пріоритетність – великий незадоволений попит
- ✓ Contested (Score 8–12): є конкуренція – потрібна диференціація
- ✓ Overserved (High Importance + High Satisfaction): ринок насичений
- ✓ Low Importance: низька пріоритетність незалежно від задоволеності

3. Аналіз матриці

Побудуйте матрицю Importance (вісь Y) vs Satisfaction (вісь X) з 10 Outcomes як точками. Позначте чотири зони кольорами. Скриншот матриці включіть до звіту.

КОПІЯ

Висновки Opportunity Scoring

Поле	Що вказати
Underserved зони (Score > 12)	*Перелік Outcomes + чому вони виявились незадоволеними?*
Overserved зони	*Яких функцій уникати? Де вже є хороші рішення?*
Порівняння з RICE та Kano	*Чи підтверджують три методи одні й ті самі пріоритети?*
Головний інсайт	*Що несподіваного виявив Opportunity Scoring?*

КОПІЯ

Завдання 5. Фінальний пріоритизований беклог

Синтезуйте результати всіх трьох методів (RICE + Kano + Opportunity Scoring) та сформуйте фінальний топ-15 пріоритизований беклог для MVP:

Ранг	Story ID	Назва Story	RICE Score	Kano	Opp. Score	Рішення (MVP/R2/Backlog/□)	Обґрунтування (1 речення)
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							
12							
13							
14							
15							

Значення колонки «Рішення»:
MVP – включити в першу версію: Must-be за Kano + High RICE + Underserved Opportunity
Release 2 – наступний реліз: Should Have + Mid RICE + Contested Opportunity
Backlog – залишити для майбутнього: Could Have + Low RICE
□ **Відкласти –** не будувати зараз: Won't Have, Indifferent за Kano, Overserved

Побудуйте User Story Map у Miro (шаблон «Story Map»): розмістіть Stories по горизонталі (хронологія User Journey) та по вертикалі (пріоритет). Виділіть горизонтальний «реліз-розріз» для MVP. Це найнаочніший спосіб комунікувати беклог стейкхолдерам.

Korlr

Вимоги до звіту

Структура звіту

Титульна сторінка (назва дисципліни, номер та назва ПР, ПІБ, група, дата)
Зміст (автоматично згенерований у Word)
Розділ 1: Ієрархія беклогу (таблиця Epics, Features, 25+ User Stories із AC у форматі Given-When-Then)
Розділ 2: RICE-пріоритизація (таблиця 15 Stories + аналіз 4 питань)
Розділ 3: Kano-аналіз (опитувальник, зведена таблиця результатів, висновки)
Розділ 4: Opportunity Scoring (таблиця 10 Outcomes, матриця зон, висновки)
Розділ 5: Фінальний беклог топ-15 з обґрунтуванням кожного рішення
Висновки: як пріоритизація змінила початкові уявлення про продукт?
Список джерел

Чек-лист перед здачею

✓	Пункт перевірки	Статус
☐	3–4 Epics – кожен прив'язаний до конкретного JTBD з ПР №01–02	
☐	6–9 Features – кожна має самостійну цінність та батьківський Epic	
☐	25+ User Stories у форматі «Як [Persona], я хочу [дія], щоб [вигода]»	
☐	Кожна Story: ID, Feature, Тип, SP (Фібоначчі), MoSCoW-категорія	
☐	Кожна Story: прив'язка до конкретного JTBD або Pain (обов'язково!)	
☐	Кожна Story: мінімум 3 AC у форматі Given-When-Then (не просто чек-лист)	
☐	Перші 5 Stories: заповнені всі поля включно з Notes та залежностями	
☐	Definition of Done: сформульований та однаковий для всіх Stories	
☐	RICE для 15 Stories: Reach (числа), Impact (шкала), Confidence (%), Effort (людино-міс)	
☐	RICE: формула коректна, відсортовано від найвищого до найнижчого	
☐	Аналіз RICE: відповіді на всі 4 питання – конкретно, з посиланням на числа	
☐	Kano: 10 функцій × мінімум 10 респондентів × 2 запитання = таблиця відповідей	
☐	Kano: використана таблиця класифікації AMOIRQ для визначення категорії	
☐	Kano: висновки включають Must-be, Performance, Delighter, Indifferent переліки	
☐	Opportunity Scoring: 10 Outcomes – НЕ функції, а бажані РЕЗУЛЬТАТИ	
☐	Opportunity Scoring: Importance та Satisfaction – середні від 10+ респондентів	
☐	Матриця зон Opportunity Scoring побудована та включена до звіту	
☐	Фінальний беклог: 15 Stories, 4 значення рішення (MVP/R2/Backlog/□)	
☐	Скриншот беклогу з Jira/Notion/Linear є у додатку до звіту	
☐	Файл названий: «ПР04_[Прізвище]_[Група].docx»	

КОПІЯ